Containerization

What is Containerization?

What is Containerization?

- Containerization is a software deployment technique where we package an application along with everything it needs to run — code, dependencies, libraries, and minimal OS files — into a single, portable unit called a container image.

What is Containerization?

- Containerization is a software deployment technique where we package an application along with everything it needs to run — code, dependencies, libraries, and minimal OS files — into a single, portable unit called a container image.

- When we run that image, it becomes a container — an isolated environment that behaves the same way on any system that supports containers.

Key Points about Containerization

- Isolation
  - Each container runs in its own space, separate from other containers and the host OS.

Key Points about Containerization

- Isolation
  - Each container runs in its own space, separate from other containers and the host OS.
- Portability & Consistency
  - Container images follow the OCI (Open Container Initiative) format.
  - "Build once, run anywhere" — works the same on a developer laptop, staging, cloud, or Kubernetes. Eliminates the "works on my machine" problem — the container has all dependencies baked in.
  - Move the app between different OS environments without code changes.

Key Points about Containerization

- Isolation
  - Each container runs in its own space, separate from other containers and the host OS.
- Portability & Consistency
  - Container images follow the OCI (Open Container Initiative) format.
  - "Build once, run anywhere" — works the same on a developer laptop, staging, cloud, or Kubernetes. Eliminates the "works on my machine" problem — the container has all dependencies baked in.
  - Move the app between different OS environments without code changes.
- Lightweight
  - Containers share the host OS kernel instead of running a full OS like virtual machines do.
  - Faster startup, smaller size, less resource overhead.

Key Points about Containerization

- Isolation
  - Each container runs in its own space, separate from other containers and the host OS.
- Portability & Consistency
  - Container images follow the OCI (Open Container Initiative) format.
  - "Build once, run anywhere" — works the same on a developer laptop, staging, cloud, or Kubernetes. Eliminates the "works on my machine" problem — the container has all dependencies baked in.
  - Move the app between different OS environments without code changes.
- Lightweight
  - Containers share the host OS kernel instead of running a full OS like virtual machines do.
  - Faster startup, smaller size, less resource overhead.
- Simplified Deployment
  - Package the app and all dependencies into a single image.
  - Deploy anywhere the container runtime is available (cloud, on-premises, local).

Key Points about Containerization

- Scalability
  - Easily run multiple containers (horizontal scaling).
  - Works well with orchestration tools like Kubernetes.

Key Points about Containerization

- Scalability
  - Easily run multiple containers (horizontal scaling).
  - Works well with orchestration tools like Kubernetes.
- Easier CI/CD Integration
  - Automate builds, tests, and deployments in pipelines.

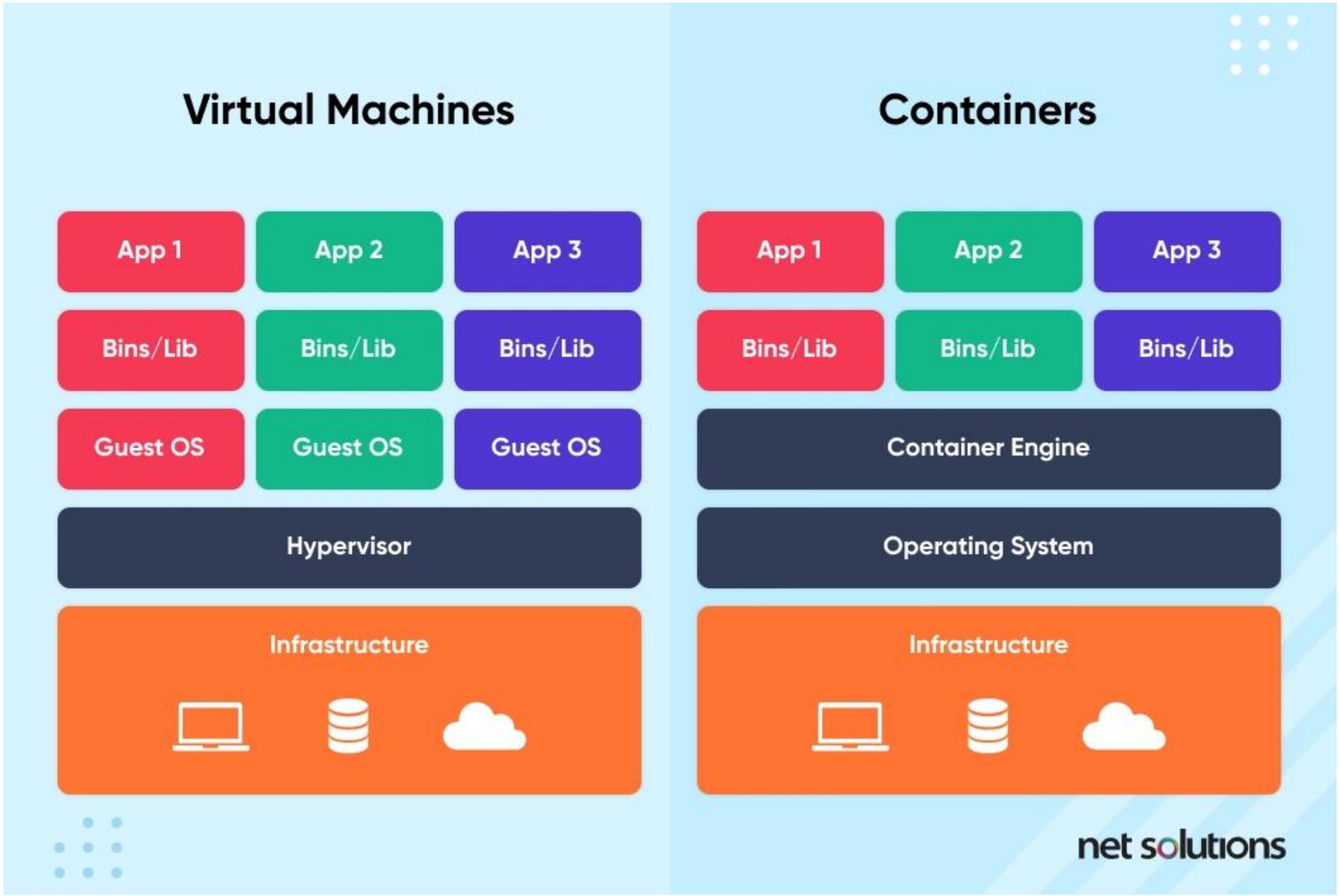
Key Points about Containerization

- Scalability
  - Easily run multiple containers (horizontal scaling).
  - Works well with orchestration tools like Kubernetes.
- Easier CI/CD Integration
  - Automate builds, tests, and deployments in pipelines.
- Security
  - Limit the attack surface by controlling the container environment.

Key Points about Containerization

- Analogy
  - Without containerization → Shipping a product by sending all parts separately and asking the customer to assemble it.
  - With containerization → Shipping a fully assembled, sealed product that works out of the box.

Infrastructure Virtualization & Containerization



Source: https://www.netsolutions.com/insights/containerization-vs-virtualization/

Infrastructure Virtualization & Containerization

The figure assumes Type-1 (bare-metal) hypervisors, like VMware ESXi or Microsoft Hyper-V run directly on hardware without a traditional Host OS.

Alternatively, Type-2 hypervisors, (such as VirtualBox or VMware Workstation); where the Host OS must be present, and it's indeed a critical layer.

Containers require a Linux kernel — always.

To run, containers on Linux, we need a Docker Engine (Docker Desktop is also an option, which includes Docker Engine and a graphical user interface.), while on Windows, we need WSL2 or Hyper-V and Docker Engine at minimal (typical choice is Docker Desktop which include Docker Engine, Docker CLI, Docker Compose, GUI, etc.).

Docker Engine is one possible container runtime. Other options are containerd, podman, etc..

The container engine (also called a container runtime) is the software that runs and manages containers by abstracting OS-level virtualization.

Infrastructure Virtualization & Containerization

Virtual Machines	Containers
• VM provides Hardware-level virtualization • Virtual Machines (VMs) simulate entire machines (hardware + OS) • Each VM runs its own OS, requires hypervisor (e.g., VMware, Hyper-V) • Useful for full isolation, but heavyweight and slower to start	• Containers provide OS-level virtualization • Container includes application code, application dependencies (e.g., Python, TensorFlow, libraries), User-space OS files (e.g., /bin, /lib, /usr from a base image like Ubuntu or Alpine), but not the full OS. • Container does not include the OS kernel (e.g., Linux kernel), low-level drivers or system services (e.g., systemd). It shares the host OS kernel • Lightweight, portable, and fast to start • Can run multiple containers on one VM or physical server

Tools to Build Container Images

Tool	Needs Daemon?	Supported Base Images	Supports Dockerfile
Docker CLI	Yes – Docker Daemon (dockerd)	Any	Yes
Podman	No	Any	Yes
Buildah	No	Any	Yes
Kaniko	No	Any	Yes
Jib	No	Java only	No
Buildpacks	No	Language-Specific/Language-Aware	No

- A **daemon** is a **background process** that runs continuously and performs tasks **without direct user interaction**.
- "Supported base images" means the set of images that a container build tool knows how to use as the starting point for building our app's container.
- For Jib → only Java runtimes (e.g., OpenJDK, distroless Java, Temurin). For general Docker builds → basically **any Linux-based image**.

More on Supported Base Images

In normal computing, a distro means “kernel + userland.”

In containers, a distro base image means “userland part of that distro,” because the kernel is always provided by the host.

In containers, the difference between a distro image and a distroless image mainly comes down to what userspace components are included. — and neither includes a kernel.

More on Supported Base Images

Distro Image (e.g., Ubuntu): It includes a full Linux distribution with shell (e.g., bash), package managers (e.g., apt), and many general-purpose tools and software.

Distroless Image (e.g., gcr.io/distroless/java17): This is a stripped-down version of a distribution. It only contains the minimal filesystem and the required libraries or runtimes (like JVM or Python). It doesn't include a shell or package manager (e.g., no apt), making it less flexible for installation or running commands but more lightweight.

Distroless images are smaller, faster, and more secure but lack flexibility (e.g., no apt-get install).

Summary: A "distro" refers to a complete Linux distribution (like Ubuntu, Debian, Alpine, etc.), whereas a "distroless" image is minimal and designed for containers with just the necessary runtime environment.

More on Supported Base Images

What is a base image?

- A base image is the starting point of our container's filesystem.

- For example:
  ubuntu:22.04 → provides userland Ubuntu Linux environment.
  python:3.11 → provides Python already installed on top of a Linux distro.
  openjdk:17 → provides a JDK preinstalled.

- When we build a container, we usually write in our Dockerfile:

```
FROM python:3.11
COPY app.py /app
CMD ["python", "app.py"]
```

  Here, python:3.11 is the base image.

Tools to Build Container Images

Tool	Learning Curve	Portability
Docker CLI	Easy	High
Podman	Moderate	High
Buildah	Moderate	High
Kaniko	Easy	Very High
Jib	Moderate	Very High
Buildpacks	Easy	Very High

Tools to Run Container Images

Tool	Needs Daemon?	Rootless Support
Docker Engine	Yes – Docker Daemon (dockerd)	Limited
Podman	No	Yes
containerd + nerdctl	Yes – containerd Daemon	Limited
CRI-O	Yes – CRI-O Daemon	Limited
Kubernetes	Needs container runtime daemon (containerd / CRI-O)	Limited

Rootless support in Docker (or containers generally) refers to running containers without requiring root (superuser) privileges -- not just inside the container, but also on the host system.

Why It Matters

Traditionally, Docker runs as the root user on the host. This poses security risks:

- A vulnerability inside a container could allow privilege escalation to the host.
- Malicious containers could compromise the entire system.

- With rootless containers, users can:
  - Run containers entirely in user space (no sudo needed)
  - Improve isolation and security
  - Use Docker in multi-user environments (like shared servers or dev boxes)

Tools to Run Container Images

Tool	Ease of Use	Speed
Docker Engine	Easy	Fast
Podman	Moderate	Fast
containerd + nerdctl	Moderate	Fast
CRI-O	Moderate	Fast
Kubernetes	Complex	Slower (orchestration overhead)

More on Tools

Method	Works Without Dockerfile?	Supports Multi-Stage Builds?	Integration with Kubernetes
Docker	No	Yes	(via Docker → containerd)
Podman	No	Yes	(direct with containerd/CRI-O)
Buildah	No	Yes	(push to registry, then deploy)
Kaniko	No	Yes	(K8s-native)
Jib (Java)	Yes	Limited	Yes
Buildpacks	Yes	Limited	Yes

Note: Docker builds the image; Kubernetes nodes typically use containerd or another CRI-compatible runtime instead of Docker. Kubernetes no longer uses the Docker Engine software directly as its internal container runtime. When people say Kubernetes "dropped" Docker, they mean Kubernetes removed native support for the Docker Engine.

More on Tools

Method	Storage Overhead
Docker	Medium
Podman	Medium
Buildah	Low
Kaniko	Low
Jib (Java)	Very Low
Buildpacks	Very Low

OCI Compliance

If a tool is OCI Image Spec–compliant, it means:

- If it's a builder → it can produce images in the OCI format.
- If it's a runner → it can pull and run images in the OCI format.

The image works across any other OCI-compliant tool, regardless of how it was built.

Analogy:

- It's like exporting a document as a PDF -- any PDF reader (or printer) can open it, no matter which software created it.

So:

- Build with Kaniko → run with Podman.
- Build with Docker → run with Kubernetes (via containerd or CRI-O).
- Build with Jib → run with Docker, Podman, etc.

All tools discussed earlier are OCI-compliant.

Dockerize the Application

1. **Prerequisite**

- Install Docker Desktop for Windows
  - Download from: https://www.docker.com/products/docker-desktop
  - Enable WSL in Windows 11 (Look at the next slide)
  - Enable WSL 2 integration in Docker Desktop (it's faster than Hyper-V).

- Install uv package manager
  - Open PowerShell: powershell -ExecutionPolicy Bypass -Command "irm https://astral.sh/uv/install.ps1 | iex"
  - Verify: uv --version

  # pip is Python package installer (official default), while uv is a fast, modern package manager for Python from Astral, designed as a drop-in replacement for pip/poetry

Dockerize the Application

1.1 To enable WSL in Windows 11:

- Open Control Panel and navigate to Programs > Turn Windows features on or off.
- Locate and check the boxes for Virtual Machine Platform and Windows Subsystem for Linux.
- Click OK and restart our computer when prompted.
- After restarting, open PowerShell or Command Prompt as administrator. We can then install a Linux distribution (optional, if we wish to use the default WSL 2 distro).

Dockerize the Application

- 1.2 Install a Linux distro (optional, if we wish to use the default WSL 2 distro)
  - Open Microsoft Store. Search for Ubuntu (or Debian, Fedora, etc.).
  - Click Get → Install.
  - Launch it once to complete setup (username & password).
- 1.3. Enable WSL 2 in Docker Desktop
  - Open Docker Desktop.
  - Click the Settings icon (top right).
  - In the General tab: Check Use the WSL 2 based engine.
- 1.4. Turn on integration for our distro:
  - In Docker Desktop, go to: Settings → Resources → WSL Integration
  - Toggle ON for the distro we installed (e.g., Ubuntu).
  - Click Apply & Restart.
- 1.5. Confirm it's working
  - Open our Ubuntu terminal (from Start Menu).
  - Run: docker –version
  - we should see Docker's version — meaning it's now working in WSL 2.
- 1.6. We may also be required to configure DNS
  - In Docker Desktop, go to: Settings → Docker Engine -> In the configuration file, add "dns": [
    "8.8.8.8",
    "8.8.4.4"].

Dockerize the Application

2. Project Structure

Make sure our files look like this:

```
my-flask-app/
├── app.py
├── model.pkl
├── requirements.txt   # will be generated later with uv
├── templates/
│   └── index.html
└── Dockerfile, .dockerignore
```

#Dockerfile instructions are case-insensitive. Names in .dockerignore and Dockerfile are case-sensitive.

Dockerize the Application

3. Prepare Python Dependencies

Create and manage dependencies with uv

- uv init --python 3.13.5
- uv venv .venv (creates venv named .venv.)
- Activate manually using activate in ./.venv/Scripts/activate)
- uv pip list (we should see nothing)
- uv run python --version
- uv add notebook #notebook is just an example, install whatever we need for our project
- uv pip list (we should see installations now).
- To uninstall anything, we need to do uv remove name_of_the_package
- (Add any other libs we use, e.g., pandas, joblib, etc.)
- To deactivate, just use "deactivate"

Export to requirements.txt (Docker still needs this)

- uv pip freeze > requirements.txt #uv pip freeze prints a list of all installed packages in the current environment with exact versions

Dockerize the Application

4. Create Dockerfile

Example:

```
# Use official Python image (Dockerfile instructions are case-insensitive. )
FROM python:3.13-slim
#(stripped of unnecessary packages. Only the bare minimum needed to run Python. Much smaller size (~40–50% of the full image).)

# Set working directory (now dest WD is /myapp (create + cd))
WORKDIR /myapp

# Copy requirements.txt from host to container
COPY requirements.txt .

# Install dependencies
RUN pip install -r requirements.txt

# Copy app files (#first is source, and then dest)
COPY . .

# Expose Flask port (container port)
EXPOSE 5000 #(EXPOSE is metadata/documentation, not a requirement or port forwarding rule.)
#to ensure it is not misleading, keep it the same as the port in app.run() or serve()

# Command to run Flask (CMD specifies the default command that will run when we start a container from this image.)
CMD ["python", "app.py"]
```

Visualize Docker Layers

- **1. Using `docker history`**

This is the simplest built-in way.

Example:

```bash
docker history myapp:latest
```

Output:

```
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
a1b2c3d4e5f6   2 hours ago    CMD ["python","app.py"]                         0B
<missing>      2 hours ago    COPY . .                                        50kB
<missing>      2 hours ago    RUN pip install -r requirements.txt             120MB
<missing>      2 hours ago    COPY requirements.txt .                         2kB
<missing>      2 hours ago    WORKDIR /app                                    0B
<missing>      3 days ago     FROM python:3.13-slim                           45MB
```

- Each line = one **layer**.
- **CREATED BY** = which Dockerfile instruction made it.
- **SIZE** = how much extra size that layer added.

Dockerize the Application

5. Build Docker Image

- From project root: docker build -t flask-ml-app . (or docker build -f dfile -t myapp:dev .)

#Creating Docker image from a Dockerfile: docker build -t name_we_want_to_give:tag_we_want_to_give . (. Refers to Dockerfile in the current directory) (while building the Docker, venv folder needs to be ignored by means of the .dockerignore file). -t labels the resulting image with a human-friendly name and optionally a tag so we can easily reference it later. Without -t, Docker still builds the image, but it will only have an image ID. The default tag is latest.

6. Run Container Locally

- docker run -p 5000:5000 flask-ml-app #host_port:container_port

Then open: http://localhost:5000 #host port
#docker run --name name_we_want_to_give_to_container -d -p host_port:container_port image_name:tag_name (-d for the detached mode)
#if we don't give a name to the container, Docker will automatically give the name to it.
#While running from Docker Desktop, don't forget to give host port to make it accessible from the host machine.

7. Stop and Remove

- docker ps # find container ID
- docker stop <id or name>
- docker rm <id or name>

Dockerize the Application

- A Dockerfile should be named exactly Dockerfile (with capital D and no extension) if we want Docker to pick it up by default. But we can use any name if we pass it explicitly with -f during docker build.
- Teams often keep multiple Dockerfiles in a project:
  - Dockerfile → production build
  - Dockerfile.dev → developer environment
  - Dockerfile.test → CI builds
- Docker will only auto-detect Dockerfile; anything else requires -f.
  - docker build -f Dockerfile.dev -t myapp:dev .
  - docker build -f Dockerfile.test -t myapp:test .
- .dockerignore has to be named exactly that way
- docker history image_name → best for seeing the actual base image lineage.
- docker run --rm <image_name> cat /etc/os-release → quick way to see which distro userland (part of the actual FROM base image) the image provides (if any). (--rm Remove the container automatically when it exits.). We can also use inspect option in images in Docker Desktop. Here, we have used "docker run <image_name> <command>" syntax to override image's default command when we start a container from the image.
- docker images prune - to prune the build that stopped in between due to errors.

Dockerize the Application

- On Windows with Docker Desktop + WSL2 backend (default setup as of 2025):
  - When we run: docker build -t myimage .
  - The image is stored inside a WSL2 virtual machine, not directly on our Windows file system.

- So What Can we Do With that?
  - we cannot copy a built image by grabbing this file — it's not usable directly.
  - we must use Docker commands like docker save to export a usable .tar image.
  - docker save -o myimage.tar imagename:tag (tar file is saved in WD)
  - docker save by default writes the image tarball to standard output (the console) (it doesn't automatically create a file — it writes the tarball data to standard output (stdout).).
  - Using -o <filename> tells Docker to write the image to the specified file instead.
  - On the other machine: docker load -i myimage.tar. This unpacks the image into Docker's local image store. –i says "Load the image from this file (myimage.tar) on disk, not from stdin." Now, we can use docker images, docker run.
  - The :latest tag is the default everywhere in Docker — build, run, save, load, push, pull — unless we specify some other tag.

Docker Network

- A Docker network allows container(s) to communicate to itself, with each other and/or with the host machine or the internet.
- By default, Docker creates networks automatically (named bridge, and the type is also bridge), but we can also create custom networks.
- On Windows, Docker networking works the same way as on Linux, but network interfaces live inside the WSL2 VM.

Docker Network

- A Docker network allows containers to communicate with each other and/or with the host machine or the internet.
  - http://host.docker.internal:3000 is typically used inside containers (like Docker containers) to access a service running on the host machine. When our code runs inside a Docker container, localhost refers to the container itself, not our computer. So Docker provides a special hostname: host.docker.internal, which points to the host machine (our laptop/server).
  - Example: Inside a container, do http://host.docker.internal:3000, which means connect to port 3000 on the host machine from inside the container. Inside a container, within a .py file, we can do:

    import requests
    requests.get("http://host.docker.internal:3000"). (or requests.post())

  - So, Inside Docker: 1. localhost → container, 2. host.docker.internal → host machine
  - Example

URL	Where it points
localhost:3000	container itself
host.docker.internal:3000	host machine

Dockerize the Microservices

```
docker network create iris_net #bridge network is created
docker build -t dbapp_image ./dbapp
docker run -d --name cntdbapp --network iris_net -p 8001:8001 dbapp_image
docker build -t webapp_image ./webapp
docker run -d --name cntwebapp --network iris_net -p 8000:8000 webapp_image
```

Notes:

1. The container port for the two containers above could have been the same. The host port must be different. We are assuming that both containers are on the same network.

2. Important: Containers on the same network, that is, if containers communicate inside a Docker network (like iris_net), host ports are not needed at all. Example: http://cntdbapp:8001. Docker resolves container names using its internal DNS. Other example response = requests.get("http://cntdbapp:8001"). This is the line webapp.py.

3. The --ip option in docker run lets you assign a specific IP address to a container, but it only works when the container is attached to a user-defined network (not the default bridge network). Works only with user-defined bridge networks. The IP must be within the subnet of the network. You cannot set --ip on the default bridge network.

Docker Network

- Why Docker Networks?
  - Container-to-container communication
    - Example: A web app container talking to a database container.
  - Isolation (More details on the next slide)
    - Containers on different networks cannot see each other unless explicitly connected.
  - Custom IP and DNS resolution
    - Containers can use container names as hostnames instead of IPs. (not on the default bridge network, but on a custom bridge network)
  - Port mapping to host
    - Allow outside access via -p or --publish.

Docker Network

- Types of Docker Networks
  - bridge (default)
    - Default network for standalone containers.
    - Containers can communicate if on the same bridge.
    - Example: docker run -d --name web --network bridge nginx (writing "--network bridge" is optional in this case, as bridge is default.)
    - Syntax: docker run --network <network_name> <image_name>. (network must have already been created)
  - host (Windows behaves differently)
    - Container shares the host's network stack.
    - On Windows, host network mode is limited / mostly ignored.
  - none
    - Container has no network.
  - Overlay
    - Used for multi-host networking in Docker Swarm (production).
  - Macvlan
    - Assigns a real MAC address to the container for advanced networking.

Docker Network

- Why create a custom bridge network instead of using the default bridge?
  - Default bridge network limitations:
    - Containers cannot communicate by name on the default bridge — only by IP address. No built-in DNS-based service discovery.
    - Limited isolation — all containers on the default bridge share the same network.
  - Custom bridge networks provide:
    - Automatic DNS resolution: Containers can reach each other by container name (e.g., db instead of IP).
    - Better isolation: we can create multiple custom networks for different projects or parts of our app.
    - More control: we can configure subnet, gateway, and other settings.
    - Simpler scaling: Easier to add/remove containers without worrying about IP addresses.

None network is useful for containers that don't need network access or for testing isolated environments (docker run --network none <image_name>). That means containers attached to this network cannot communicate with anything: No internet, No other containers, No host machine, No DNS. The container only has the loopback interface (localhost).

If we don't specify any network, Docker connects the container to the default bridge network (bridge).

The default bridge allows basic networking with NAT, but:

- containers on the default bridge cannot resolve each other by name (only by IP).
- They can access the internet through NAT.

Docker Network

- Docker comes with three built-in networks

Network Name	Network Type (Driver)	Description
bridge	bridge	Default network for containers
host	host	Container shares host network
none	null/none	No network connectivity

Docker Network

- Containers communicate over the container network using container ports, not the mapped host ports.
- The port mapping, like 9001:8001 is for exposing the container's port to the host, not for container-to-container communication.
- Port 9001 is exposed on the host machine, so external apps (like your laptop or server) can reach container on port 9001. Inside the Docker network, containers communicate using internal ports, i.e., the ports the apps listen on inside the containers (8001, 8002). The port mappings (9001:8001) are only for host-to-container traffic.

Who is making the request?	Address and Port to Use
External client on the host	localhost:9001
Container 2 (app2) → Container 1 (app1)	container1-name-or-service:8001

Docker Network

Command	Type Created	Notes
docker network create mynet	**bridge**	Custom bridge, works for container-to-container communication on a single host
docker network create --driver host mynet	host	Container shares host's network stack (limited on Windows)
docker network create --driver overlay mynet	overlay	Used for multi-host networks in Docker Swarm
docker network create --driver macvlan mynet	macvlan	Advanced: container gets its own MAC & IP on LAN
docker network create --driver none mynet	none	No networking

Docker Network

Command	Purpose	Details / Notes	Example
docker network ls	List all networks	Shows default and custom networks on the host	docker network ls
docker network inspect <name>	Show detailed info about a network	Includes subnet, gateway, connected containers, and IP addresses	docker network inspect bridge
docker network create <name>	Create a custom network	Default driver is **bridge**; can specify --driver for other types	docker network create mynet
docker run --network <name> ...	Attach container to a network at creation	Ensures container joins specified network instead of default bridge	docker run -d --name web --network mynet nginx
docker network connect <network_name> <container_name>	Connect existing container to a network	Useful for adding containers to multiple networks	docker network connect mynet web
docker network disconnect <network_name> <container_name>	Disconnect container from a network	Removes network connectivity without stopping container	docker network disconnect mynet web
docker network rm <network_name>	Remove a network	Cannot remove a network if containers are attached	docker network rm mynet

Docker Compose

What is Docker Compose?

- Docker Compose is a tool to define and manage multi-container applications using a single YAML file (docker-compose.yml).

- Instead of running multiple docker run commands, we can start, stop, and manage all containers at once.

- The docker-compose.yaml file should be placed in the root directory of our project, alongside our app folders and Dockerfiles.

Docker Compose

What is Docker Compose?

- When we run docker-compose up (older, based on Docker Compose V1, it was a standalone tool, installed separately) / docker compose up (newer, based on Docker Compose V2, integrated directly into the Docker CLI), Docker automatically looks for docker-compose.yml (or docker-compose.yaml) in the current directory. If we have given a different name (e.g. myapp.yml), use docker-compose -f **myapp.yml** up

- Docker Compose accepts both extensions: docker-compose.yml, docker-compose.yaml

Docker Compose

Key Benefits:

- Service-based configuration – environment variables, ports, volumes, dependencies, etc.
- Single command – docker-compose up brings the entire app online.

Docker Compose

Sample docker compose file (It will create a network automatically, and all containers will be in this network):

```
version: '3.9'  # or just '3'
services:
  dbapp:
    image: dbapp:v1
    container_name: dbapp
    volumes:
      - dbdata:/app/data  # data to be persisted
    ports:
      - "8001:8001"
    networks:
      - appnet
  webapp:
    image: webapp:v1
    container_name: webapp
    ports:
      - "5000:5000"
    networks:
      - appnet
    depends_on:
      - dbapp
    environment:
      - DB_URL=http://dbapp:8001  # use container name as hostname
volumes:
  dbdata:
networks:
  appnet:
```

Docker Compose

Section	Description	Mandatory?
version	Specifies file format version; recommended only for backward compatibility	Optional (for modern Compose versions)
services	Defines the containers (services) to run	**Yes** (at least one service)
networks	Defines custom networks for services to communicate	No (optional)
volumes	Defines named volumes to persist data	No (optional)
configs	Defines configuration data (for Docker Swarm mode)	No (optional)
secrets	Defines secrets for services (Docker Swarm mode)	No (optional)
build	(inside a service) Defines how to build an image	No (optional, inside service)
depends_on	(inside a service) Specifies service startup order	No (optional, inside service)

- There can be extensions - Custom extensions and metadata (rarely used) – Optional.
- Under a service, we can have build, container_name, volumes, ports, networks, image, healthcheck, depends_on, environment (Plus many more like: command (Override the default command.), …).
- Under build, we have 'context'.

Docker Compose

- Here are some additional keys we can use under a service in a docker-compose.yml:
  - command — Override the default command.
  - entrypoint — Override the default entrypoint.
  - restart — Restart policy (no, always, on-failure, unless-stopped).
  - logging — Configure logging drivers and options.
  - extra_hosts — Add host-to-IP mappings.
  - depends_on — We mentioned this; controls startup order.
  - secrets — Manage Docker secrets.
  - configs — Use Docker configs (usually with swarm mode).
  - deploy — Deployment configurations like replicas, resource limits (used with swarm mode).
  - user — Run the container as a specific user.
  - privileged — Run container in privileged mode.
  - dns / dns_search — Configure DNS settings.
  - tmpfs — Mount temporary filesystems.
  - healthcheck — We mentioned this.
  - shm_size — Size of /dev/shm.
  - stop_grace_period — Time to wait before forcefully stopping.
  - pid — PID namespace to use.
  - ulimits — Configure user limits.
  - cap_add / cap_drop — Add/drop Linux capabilities.
  - isolation — Windows container isolation mode.
  - runtime — Runtime to use (e.g., runc).

Docker Compose

The content of docker-compose.yml is case-sensitive. YAML uses indentation to define structure, not braces like JSON or XML. Indentation must be consistent -- usually 2 spaces or 4 spaces (never tabs).

Docker Compose

The content of docker-compose.yml is case-sensitive. YAML uses indentation to define structure, not braces like JSON or XML. Indentation must be consistent -- usually 2 spaces or 4 spaces (never tabs).

Docker Compose Service Name Rules:

- Service names must be unique within the services: block and should consist only of alphanumeric characters, dashes (-), and underscores (_).
- While uppercase letters are technically permitted, it is a best practice to use all lowercase because service names function as hostnames for internal DNS resolution, which is case-insensitive.
- You should avoid using spaces or special symbols like dots and symbols.
- While matching the service name to its folder name can provide clarity, it is more important to name the service after its specific function (e.g., db or web-api) to keep your configuration intuitive.

Docker Compose

The content of docker-compose.yml is case-sensitive. YAML uses indentation to define structure, not braces like JSON or XML. Indentation must be consistent -- usually 2 spaces or 4 spaces (never tabs).

Docker Compose Service Name Rules:

- Service names must be unique within the services: block and should consist only of alphanumeric characters, dashes (-), and underscores (_).
- While uppercase letters are technically permitted, it is a best practice to use all lowercase because service names function as hostnames for internal DNS resolution, which is case-insensitive.
- You should avoid using spaces or special symbols like dots and symbols.
- While matching the service name to its folder name can provide clarity, it is more important to name the service after its specific function (e.g., db or web-api) to keep your configuration intuitive.

depends_on controls container start order, not readiness. As an example, depends_on will ensure that dbapp starts before webapp starts, but it does NOT wait for dbapp to be ready — only that it's been started.

Docker Compose

The content of docker-compose.yml is case-sensitive. YAML uses indentation to define structure, not braces like JSON or XML. Indentation must be consistent -- usually 2 spaces or 4 spaces (never tabs).

Docker Compose Service Name Rules:

- Service names must be unique within the services: block and should consist only of alphanumeric characters, dashes (-), and underscores (_).
- While uppercase letters are technically permitted, it is a best practice to use all lowercase because service names function as hostnames for internal DNS resolution, which is case-insensitive.
- You should avoid using spaces or special symbols like dots and symbols.
- While matching the service name to its folder name can provide clarity, it is more important to name the service after its specific function (e.g., db or web-api) to keep your configuration intuitive.

depends_on controls container start order, not readiness. As an example, depends_on will ensure that dbapp starts before webapp starts, but it does NOT wait for dbapp to be ready — only that it's been started.

Without depends_on, services are started in the order they are defined in the docker-compose.yml file, but there is no strict guarantee on the order. If startup order matters (e.g., database before app), use depends_on, but note it doesn't wait for readiness. Health checks can ensure that a service is fully ready before other services try to interact with it.

Docker Compose

- In Docker Compose, the ports setting uses the format "HOST_PORT:CONTAINER_PORT", where the host port is exposed on our machine and the container port is the one our app listens to inside the container.

- These ports can be the same (e.g., 5000:5000) or different (e.g., 8080:5000, which means our app runs on port 5000 inside the container but is accessible on http://localhost:8080 from our browser).

- The container port must match the port our app is configured to use; otherwise, the app won't respond (e.g., 5000:8000 will fail if the app listens on 5000, not 8000).

- Always ensure the container port matches the internal app port, while the host port can be chosen based on availability or preference.

Docker Compose

YAML: When to use dash (-) and when not to

```
#Use `-` when the value is a list (multiple items)
ports:
  - "5000:5000"
  - "8000:5000"

volumes:
  - dbdata:/data
  - logs:/logs

#Do NOT use `-` for single key-value pairs
image: myapp:latest
container_name: webapp
restart: always

# we can also define lists with key-value pairs like this:
environment:
  - DEBUG=true
  - SECRET_KEY=abc123

#Or use map-style syntax (recommended for env vars)
environment:
  DEBUG: true
  SECRET_KEY: abc123
```

Docker Compose

Here's a docker_compose.yaml file that **builds the images** for both webapp and dbapp from local Dockerfiles (instead of prebuilt images)

```
version: '3.9'
services:
  dbapp:
    build:
      context: ./dbapp
    container_name: dbapp
    volumes:
      - dbdata:/app/data  # data to be persisted
    ports:
      - "8001:8001"
    networks:
      - appnet
  webapp:
    build:
      context: ./webapp
    container_name: webapp
    ports:
      - "5000:5000"
    depends_on:
      - dbapp
    networks:
      - appnet
    environment:
      - DB_URL=http://dbapp:8001  # Service name used as hostname
volumes:
  dbdata:
networks:
  appnet:
```

docker compose up --build –d
This will:

- Build images using Dockerfiles in webapp/ and dbapp/
- Create containers
- Connect them via appnet network
- Mount a volume (dbdata) for persistent storage
- Start containers

- --build is in action, only when there is a build subsection. It guarantees that context is always checked for any changes ensuring stale image is never used.

Here, we have not specified image: as a sibling to build:, and therefore, the image name will be auto-generated.

Please note, both bind mount and named volume can be created through docker compose: mydata:/app/data → Docker-managed named volume. ./hostfolder:/app/config → bind mount from host directory

Docker Compose

Regarding the previous two docker-compose.yaml file:

Limitation:

- We must ensure that the name of the "db container" is the same as used in the code (webapp/app.py).
- We can slightly decouple this with the help of 1. environment variable: docker run -e evar1=value (e.g. "DB_HOST"="other_container_name") … imgwebapp:v1. 2. docker compose and yaml file. Neither way can make it completely independent.
- It is just that in the app.py of webapp, we can use an environment variable instead of the container name, but the environment variable name and the name in the app.py must match. Also, the value of the environment variable must be the name of the db_container.

Docker Compose

Key Docker Compose Commands

Command	Purpose	Notes / Example
docker-compose up	Build (if 1. build section exists and 2. image doesn't exist) and start all services. docker-compose up --build will lead to a confirmed build check. (all or specific)	docker-compose up -d → runs in background
docker-compose down	Stop and remove containers, networks (volumes & images are optional) (**everywhere,** managed by the current compose project) (all option only)	docker-compose down
docker-compose build	Build/rebuild images for services (just build, no start) (always builds) (managed by the current compose project) (all or specific)	docker-compose build web (just build, no start)
docker-compose ps <service_name>/-a	Shows all (running + stopped) containers (managed by the current compose project). (all or specific) (status +key information)	docker-compose ps
docker-compose logs	Show logs for all or a specific service (managed by the current compose project)	docker-compose logs db
docker-compose stop	Stop running services (does not remove) (all or specific managed by the current compose project)	docker-compose stop
docker-compose restart	Restart services (all or specific) (stop + start). We have docker-compose start, which just starts containers. (managed by the current compose project)	docker-compose restart web

Docker Compose

Key Docker Compose Commands

Compose file	docker compose up behavior
build: only, **no** image:	Runs the build step, if image does not exist and there is no change in the build context; uses autogenerated image name (usually projectname_servicename) (so, actually, not random)
build: + image: specified	Builds **only if image doesn't exist locally**, or if --build is used, it leads to guarantted context check
image: only, **no** build:	Pulls existing image from registry/locally. –build is ignored.

Remove volumes as well as images in docker compose down:

- Example: docker-compose down --rmi all --volumes
- Note: We cannot remove specific volume or image using docker compose down

Flag	What it removes
--volumes	Removes volumes associated with services (defined in volumes: or anonymous)
--rmi all	Removes **all images used by the services**, both locally built **and** pulled images
--rmi local	Removes **only images built locally** via the build: section; **does not** remove pulled images

Export-Import and Save-Load

- Docker Export (container to archive):
  - Input: A container (running or stopped)
  - Output: A .tar archive of the container's filesystem
  - It does not include: Image layers, History, Metadata like ENV, CMD, EXPOSE, Volumes or bind mounts, etc. It gives you a "flattened" version of the container's current state.
  - Syntax: docker export <container_name_or_id> > some_file.tar
  - The above exports the filesystem of the container (not the image). The output is a .tar file containing the container's current state (like a snapshot of its disk).
  - Example: docker export my_container > my_container.tar

Export-Import and Save-Load

- Docker Import (archive to image):
  - Input: A .tar file (like from docker export)
  - Output: A new Docker image
  - The image has: Only the filesystem from the tarball. Optionally, a tag and some basic metadata
  - From that image, we can run new containers, but they'll be missing the original container's config (unless you manually add it).
  - Syntax: docker import some_file.tar recovered-image
  - Example: docker import my_container.tar my-recovered-image
  - Now, we can run a container like: docker run my-recovered-image

Export-Import and Save-Load

- For images, use docker save/docker load instead:
  - docker save my-image:tag > image.tar or docker save -o myimage.tar imagename:tag
  - docker load < image.tar or docker load -i myimage.tar
  - save/load are for images, and they preserve more data. Export/import is for containers, but it loses metadata and history.

Export-Import and Save-Load

- If we've used docker export → docker import, we end up with a new image, but it's "barebones" -- missing metadata like: CMD, ENV, EXPOSE, VOLUME, ENTRYPOINT, Labels, etc.

- So when we run a container from that image, we'll likely want to specify or restore that metadata manually.

Export-Import and Save-Load

- Option 1: Specify metadata at docker import time
  - You can pass metadata using the --change (-c) flag during docker import.
  - Example: docker import -c "CMD [\"python3\"]" -c "ENV DEBUG=true" my_container.tar my-image:latest
  - This sets the default command to python3
  - Sets an environment variable DEBUG=true
  - We can pass multiple -c flags
  - Supported --change flags include: CMD, ENTRYPOINT, ENV, EXPOSE, VOLUME, LABEL, WORKDIR, USER
  - Another example: docker import -c "ENV VAR1=value1" -c "ENV VAR2=value2" -c "CMD [\"python3\"]" my_container.tar my-image:latest

Export-Import and Save-Load

- Option 2: Use docker run to specify config manually
  - If you didn't set it during import, you can still add config at runtime:
    - docker run -it -e DEBUG=true -p 8080:80 --volume /data my-image:latest /start.sh
    - -e sets environment vars
    - -p maps ports
    - --volume adds volumes
    - Final argument (/start.sh) is the command (overrides CMD)
  - But this is manual every time unless you script it.

Export-Import and Save-Load

- Option 3: Wrap the imported image in a new Dockerfile
  - If we want to make our imported image more permanent and easier to reuse:
    - #Dockerfile
    - FROM my-imported-image
    - ENV DEBUG=true
    - CMD ["/start.sh"]
    - EXPOSE 80
  - Then build a new image: docker build -t my-restored-image .

Docker Volume

- Containers are ephemeral — meaning if we remove a container, all data inside it is lost.
- If a container is stopped and started again, the data inside it is not lost, even if no volume is used, because the container's writable layer still exists. However, if the container is removed or recreated, the data is lost without a volume since the container filesystem is deleted. When a volume is used, the data is stored outside the container, so it persists even if the container is removed or recreated. A Docker volume is a storage mechanism that allows data to persist independently of the container lifecycle.
- Volumes solve this by storing data outside the container's writable layer, on the host machine (or remote storage).
- Volume = data on the host machine (outside the image/container filesystem).
- If we move a container to another system (export/import or push/pull from Docker Hub), the volume data does not go/come with it.

Docker Volume

Types of Docker Storage

- Named Volumes (recommended) (we will only see this)
  - Managed by Docker (docker volume create)
  - \\wsl$\docker-desktop-data\version-pack-data\community\docker\volumes\ (Docker Desktop must be running. Sometimes version-pack-data doesn't show until Docker has initialized at least one volume.)
  - This is a network path. We can browse in Windows Explorer.

- Bind Mounts
  - Directly map a host directory or file into a container

- tmpfs Mounts
  - Stored in RAM only
  - Disappears after the container stops

Docker Volume

Feature	Named Volume	Bind Mount (Windows folder)
Location	Inside Docker-managed VM (WSL)	Direct Windows folder (C:\...)
Persistence	Yes, survives container removal	Yes, survives container removal
Sharing between containers	Easy, multiple containers can mount the same volume	Easy, multiple containers can mount the same folder
Portability	Less portable in the sense that we **cannot just copy the container image and expect the data to move**; we need to backup & restore	More portable in the sense that the folder itself can be copied to another machine; we can then mount it to a new container
Performance (Windows)	Faster (inside VM)	Slower (sync between Windows ←→ WSL)
Isolation / Safety	Managed by Docker; safer	Direct access to host folder; more chance of accidental modification

Docker Volume

Create a named volume

- docker volume create mydata
- A bind mount is not created with docker volume create. Instead, it's defined directly when running a container by specifying a host path. (docker run -d -v mydata:/app/data -v C:\hostfolder:/app/config --name mycontainer myimage)

List all volumes

- docker volume ls
- Shows existing named/anonymous volumes (not any other types). Helps in auditing or selecting a volume. We can look at the NAME column (e.g., mydata), which is useful before cleanup.

Inspect a volume

- docker volume inspect mydata #(only for the named volume)
- Displays details like mountpoint and usage. Helps when we need the on-disk location or troubleshoot.

Docker Volume

Run a container with a named volume

- docker run -d --name web --publish 8080:80 -v mydata:/usr/share/nginx/html nginx
  - Creates (if it doesn't already exist) the named volume mydata and mounts it to /usr/share/nginx/html into the container.

- docker run -d -v /usr/share/nginx/html nginx
  - In this case, Docker will create an anonymous volume and mount it to /usr/share/nginx/html inside the container. Docker will automatically name the volume with a randomly generated name, and we won't be able to directly refer to it by a human-readable name.

  - Use case: This is useful when we just need temporary storage or don't need to refer to the volume by name after the container is removed.

Important:

- Stop/remove container → data in mydata stays.
- Reuse the same volume on new containers.

Docker Volume

Breakdown of -v mydata:/usr/share/nginx/html

Part	Where it lives	What it means
mydata	Host machine (Docker-managed VM in Windows: \\wsl$\docker-desktop-data\...)	This is the **named volume** storing persistent data. Even if the container is removed, the data in mydata stays.
/usr/share/nginx/html	Inside the container	This is the **container path** where Nginx looks for website files. The volume is "mounted" here, so anything written in this folder actually goes into the volume.

Note: Docker prioritizes the volume over the container's internal filesystem.

Docker Volume

Remove a volume (cleanup)

- To delete a named volume and its data (When we're sure the data is no longer needed). (Before we can remove a named volume, we must remove any containers that are using it.)
  - docker rm -f web # forcibly remove container (if still running)
  - docker volume rm mydata # remove named volume

- Bulk cleanup (irreversible): docker volume prune -f # removes ALL unused volumes

- docker rm -v <container> only auto-removes anonymous volumes tied to that container. Named volumes (like mydata) are not removed unless we run docker volume rm. Removes the container also.

Docker Volume – In our Microservices App

1. Create and attach volume
docker volume create voldbapp
docker run -d -p 8001:8001 --name cntdbapp --network flasknet1 -v voldbapp:/dbapp/ imgdbapp:v1
#dbapp is the name of the workdir (Here, in workdir DB is saved)
docker run -d -p 8000:8000 --name cntwebapp --network flasknet1 imgwebapp:v1

2. Use it, insert some records, and view records. Kill it.

3. Rerun it.
docker run -d -p 8001:8001 --name cntdbapp --network flasknet1 -v voldbapp:/dbapp/ imgdbapp:v1
docker run -d -p 8000:8000 --name cntwebapp --network flasknet1 imgwebapp:v1

4. You will see that the volume persists.

5. Limitation: We have to ensure that the name of the “db container” is the same as the one used in the code (as the webapp code is written that way).

Multi-Stage Build

What did we do?

Stage 1: built wheels (Python packages).
Stage 2: installed them cleanly.
Ensured the final image doesn't have leftover build tools.
Note: We can have more than two stages (Builder, Test, Final, Debug/Helper)

- Why Multi-Stage is Valuable?
  - Heavy build dependencies stay in Stage 1 → discarded.
  - Final stage of the Docker file gets only the wheels → small, fast, secure.

- What's removed in a multi-stage Docker image that still exists in a single-stage Docker image?
  - Source archives (e.g., .tar.gz) downloaded by pip install when a wheel isn't available.
  - Temporary directories used to build extensions (e.g., build/, .eggs/, etc.).
  - C/C++ build tools like gcc, g++, make, headers — pulled in implicitly by pip if building from source.
  - Python package source trees used during build (e.g., numpy/, cryptography/).

Multi-Stage Build

- What is a Multi-Stage Docker Build?
  - A multi-stage build uses multiple FROM instructions in a single Dockerfile.
  - Each FROM starts a new stage.
  - We can copy artifacts (binaries, files, etc.) from one stage to another.
  - Final stage only contains what we need → smaller, cleaner image. Doesn't ship compilers, build tools, or test dependencies to production.
  - More secure, less attack surface (fewer packages inside the final image).

Multi-Stage Build

```
# ---- Stage 1: Build dependencies ----
FROM python:3.12-slim AS builder
WORKDIR /app

COPY requirements.txt .

RUN pip wheel --no-cache-dir --wheel-dir /app/wheels -r requirements.txt
```

Multi-Stage Build

- #When Docker executes RUN pip wheel ... command, it spins up a temporary container to process that specific instruction. Everything that follows happens entirely inside this temporary container.

- #For every package listed in requirements.txt, pip wheel will first check its local cache (~/.cache/pip inside the temporary container. It is empty initially, but checking will be useful in case of shared dependencies between packages listed in requirements.txt) for a compatible wheel (not a source distribution). If no cached wheel is found, it will try to download a pre-built .whl from PyPI (or other configured indexes).

- #When downloading a pre-built wheel (assuming --no-cache-dir is used), pip temporarily downloads the .whl file into an ephemeral /tmp folder inside the temporary container before immediately moving it to the specified --wheel-dir (or to the current working directory by default, e.g., /app).

- #If a pre-built wheel is not available, pip will download the source distribution (.tar.gz or .zip) from PyPI and build a wheel from it. During this build phase, pip downloads and extracts the source distribution to /tmp, again, inside that same temporary container. It compiles the wheel there, and then moves the resulting .whl file to your target directory.

Multi-Stage Build

Crucially, whether pip downloaded a pre-built wheel or compiled one from source, it deletes the residual files from /tmp before the command finishes. Once the RUN command successfully completes, Docker takes a snapshot of the filesystem changes to create a new image layer, and then permanently destroys the temporary container. Because pip cleaned up /tmp before that snapshot was taken, those temporary download and source files are never saved to your final Docker image, ensuring they won't bloat its size.

If neither a wheel nor a source distribution is available (which is rare for well-maintained packages), pip will abort and raise an error: Could not find a version that satisfies the requirement xyz....

Finally, it is important to note that if --no-cache-dir is not used, pip will leave cached files (downloaded wheels or source archives) in its default cache directory (~/.cache/pip inside the temporary container). Because this cache folder is not automatically cleaned up like /tmp, those duplicate files will be captured in Docker's snapshot before the temporary container is destroyed, permanently baking them into that specific Docker layer.

Multi-Stage Build

Scenario A: WITH --no-cache-dir (The optimized way)

- Downloads from PyPI -> /tmp
- Moves from /tmp -> /app/wheels/
- tmp is emptied.
- Result in Docker layer: 1 copy of the wheel (in /app/wheels).

Scenario B: WITHOUT --no-cache-dir (The default, bloated way)

- Downloads from PyPI -> ~/.cache/pip
- Copies from ~/.cache/pip -> /app/wheels
- Cache is left entirely alone.
- Result in Docker layer: 2 copies of the wheel (one in /app/wheels, one hidden in ~/.cache/pip).

#In cases where some packages need to be built from source (because pre-built wheels are not available on PyPI), we may need to install system-level build tools and libraries using apt-get update and apt-get install -y before attempting to build wheels with pip wheel. Since we're using a multi-stage build, these build tools will not be included in the final image — so there's no need to worry about image bloat. It's perfectly fine (and often recommended) to preemptively install common build dependencies before running pip wheel to ensure compatibility with most packages that require compilation.

Multi-Stage Build

In case we need gcc, make, etc., we will have the following in the Docker file before COPY requirements.txt . (to take full advantage of Docker's layer caching. Placing after COPY requirements.txt . will limit the advantage of the layer caching)

```
# Update package lists and install build dependencies manually
RUN apt-get update && apt-get install -y \
    build-essential \     # includes gcc, g++, make
    gfortran \           # Fortran compiler (needed by numpy/scipy)
    python3-dev \        # Python headers
    musl-dev \           # For Alpine or specific libc-dev (if needed)
    && rm -rf /var/lib/apt/lists/*
```

rm -rf /var/lib/apt/lists/*: This is a clean-up step to reduce image size. It deletes cached package **metadata** downloaded during apt-get update. Keeps the Docker image smaller and cleaner. This is a best practice in Docker to avoid bloated layers.

Even if we clean up things like /var/lib/apt/lists/, our image still includes compilers and build tools — which we don't want in production. Multi-stage builds let us compile heavy stuff in Stage 1, throw away all build tools, and keep only what's needed to run the app.

Multi-Stage Build

```
# ---- Stage 1: Build dependencies (Builder Stage)----
FROM python:3.12-slim AS builder
WORKDIR /app

COPY requirements.txt .

RUN pip wheel --no-cache-dir --wheel-dir /app/wheels -r requirements.txt


# ---- Stage 2: Final image (Final Runtime Stage)----
FROM python:3.12-slim
WORKDIR /app

COPY --from=builder /app/wheels /wheels    #sibling here, can be /app/wheels
RUN pip install --no-cache-dir /wheels/*     #can be /app/wheels/*

COPY . .

CMD ["python", "app.py"]
```

Multi-Stage Build

In RUN pip install --no-cache-dir /wheels/*, **without** --no-cache-dir: Pip will still install the wheels from /wheels/* (local files, no downloads), but pip will create and keep cache files in its default cache directory (~/.cache/pip).

This cache may include metadata about installed packages. Copies of wheel files or other installation artifacts. These cached files remain in the Docker image layer, increasing the image size unnecessarily. Over time, this can cause Docker images to be larger than needed, which slows down image transfer and uses more disk space.

Dockerize Flask + Waitress + Nginx

#Single Stage Dockerfile

```
# Base image with Python
FROM python:3.13-slim

# Set working directory inside container
WORKDIR /app

COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy all project files (filtered by .dockerignore)
COPY . .

# Install Nginx
RUN apt-get update && apt-get install -y nginx && rm -rf /var/lib/apt/lists/*

# Move Nginx config to proper location
RUN mv /app/nginx/default.conf /etc/nginx/conf.d/default.conf

# Expose HTTP port
EXPOSE 80

# Start Nginx and Flask (via Waitress) together
CMD service nginx start && waitress-serve --host=127.0.0.1 --port=5000 app:app
```

- app:app means: Import the Python file named app.py. Use the Flask application object named app inside that file.
- In copy, the source is your host machine (build context), but the destination is inside the container.
- MV runs inside the container. The source & destination are both inside the container filesystem

Directory Structure

```
project/
│
├─ app.py
├─ model.pkl
├─ preprocessors.pkl
├─ requirements.txt
├─ index.html
├─ nginx/
│   └─ default.conf
└─ .dockerignore
```

.dockerignore

```
__pycache__
*.pyc
*.pyo
.git
*.gitignore
.env
```

Dockerize Flask + Waitress + Nginx

Start Nginx and Flask (via Waitress) together
CMD service nginx start && python app.py will also work.

In waitress-serve --host=127.0.0.1 --port=5000 app:app, it says to serve the WSGI application on the host 127.0.0.1 and port 5000. Where can we find this application? In app.py (first app in app:app), there is an application named app (second app in app:app).

Dockerize Flask + Waitress + Nginx

```
#Multi Stage Dockerfile
# Stage 1: Builder
FROM python:3.13-slim AS builder
WORKDIR /app

COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /app/wheels -r requirements.txt

# Stage 2: Final Image
FROM python:3.13-slim
WORKDIR /app

# Copy Python wheels from builder and install
COPY --from=builder /app/wheels /wheels

RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels

# Copy application files
COPY . .
```

Dockerize Flask + Waitress + Nginx

```
#Multi Stage Dockerfile
# Install Nginx
RUN apt-get update \
    && apt-get install -y nginx \
    && rm -rf /var/lib/apt/lists/*

# Move Nginx config to proper location
RUN mv /app/nginx/default.conf /etc/nginx/conf.d/default.conf

# Expose HTTP port
EXPOSE 80

# Start Nginx and Waitress together
CMD service nginx start && waitress-serve --host=127.0.0.1 --port=5000 app:app
```

More on CMD

Docker allows two ways to specify CMD:

Option A: Shell form

- CMD service nginx start && waitress-serve --host=127.0.0.1 --port=5000 app:app
- Runs in a shell (/bin/sh -c).
- Allows chaining commands with &&. && is a shell operator meaning "run the next command only if the previous command succeeds".
- 1. app (before colon) → the Python file (module) without .py. Example: our file is app.py → module name is app. 2. app (after colon) → the Flask application object inside that file
- Drawback: If any command fails, the container may not handle signals properly.

Option B: Exec / JSON form

- CMD ["waitress-serve", "--host=0.0.0.0", "--port=5000", "app:app"]
- Runs directly, bypassing shell.
- Signals (like SIGTERM) are handled better, safer in production.
- But we cannot easily start multiple commands (like service nginx start && ...) in one CMD in JSON form.

Docker Commands

Setup and Installation Check

Step	Command	Syntax	Example	Description
1	**Check if Docker is installed**	docker --version	docker --version	Verifies Docker is installed and shows version.
2	**Check Docker service status**	docker info	docker info	Shows detailed info about Docker (if the daemon is running).

Docker Commands

Image and Containers

Command	Syntax	Example	Description
Pull Image	docker pull IMAGE[:TAG]	docker pull nginx:latest	Download image from registry.
List Images	docker images	docker images	Show all local images.
Run Container	docker run [OPTIONS] IMAGE	docker run -it ubuntu bash	Start container and run command.
Run in Detached Mode	docker run -d IMAGE	docker run -d nginx	Run container in background.
Run with Port Mapping	docker run -p HOST:CONTAINER IMAGE	docker run -p 8080:80 nginx	Expose container ports.

Docker Commands

Image and Containers

Command	Syntax	Example	Description
Name a Container	docker run --name NAME IMAGE	docker run --name web nginx	Assign name to container.
List Containers	docker ps / -a	docker ps -a	Show running / all containers.
Stop Container	docker stop CONTAINER	docker stop web	Gracefully stop container.
Remove Container	docker rm CONTAINER	docker rm web	Delete container.
Build Image	docker build -t NAME .	docker build -t myapp .	Build image from Dockerfile.
Tag Image (creating alias)	docker tag SRC:TAG DEST:TAG	docker tag myapp myrepo/myapp:v1	Add new tag to image.
Push Image	docker push IMAGE (You must be authenticated)	docker push myrepo/myapp:v1	Upload image to Docker Hub. (username/reponame/tag)

Docker Commands

Networking

Command	Syntax	Example	Description
List Networks	docker network ls	docker network ls	List all Docker networks.
Create Network	docker network create NAME	docker network create my-net	Create a user-defined bridge network.
Connect Container to Network	docker network connect NETWORK CONTAINER	docker network connect my-net web	Attach container to a network.
Run Container on Custom Network	docker run --network NETWORK IMAGE	docker run --network my-net nginx	Start container on a specific network.
Inspect Network	docker network inspect NETWORK	docker network inspect bridge	Show detailed network configuration.

Docker Commands

Volumes (Persistent Data)

Command	Syntax	Example	Description
Create Volume	docker volume create NAME	docker volume create mydata	Create persistent volume.
List Volumes	docker volume ls	docker volume ls	List all volumes.
Mount Volume	docker run -v VOLUME:/path IMAGE	docker run -v mydata:/data ubuntu	Mount volume into container.
Inspect Volume	docker volume inspect NAME	docker volume inspect mydata	Get volume metadata.
Remove Volume	docker volume rm NAME	docker volume rm mydata	Delete a volume.

Docker Commands

Detached Mode & Background Containers

Command	Syntax	Example	Description
Run in Detached Mode	docker run -d IMAGE	docker run -d nginx	Runs in background (returns container ID).
Attach to Running Container	docker attach CONTAINER	docker attach web	Reattach to a running container.
Run with Restart Policy	--restart=always	docker run --restart=always nginx	Automatically restart container on failure or reboot.

Docker Commands

Docker Compose: Compose is used to define and run multi-container applications.

Command	Syntax	Example	Description
Start Services	docker-compose up	docker-compose up	Starts all services defined in docker-compose.yml.
Start in Detached Mode	docker-compose up -d	docker-compose up -d	Starts services in background.
Stop Services	docker-compose down	docker-compose down	Stops and removes containers, networks, etc.
Build Services	docker-compose build	docker-compose build	Builds images for services.
View Logs	docker-compose logs	docker-compose logs	Shows logs from all services.
Scale Services	docker-compose up --scale SERVICE=NUM	docker-compose up --scale web=3	Run multiple instances of a service.

Docker Commands

Cleanup & Maintenance

Command	Syntax	Example	Description
Prune Containers, Images, Networks	docker system prune	docker system prune -a	Remove all unused resources.
Remove All Stopped Containers	docker container prune	docker container prune	Clean up stopped containers.
Remove All Unused Volumes	docker volume prune	docker volume prune	Deletes all unused volumes.

Resource Management

Resource Management

```
docker run -d \
  --name myapp \
  --memory="512m" \              # Max memory allowed (hard limit)
  --memory-reservation="256m" \  # Soft limit (container should get at least this much)
  --cpus="1.5" \                 # Use at most 1.5 CPU cores
  --cpu-shares=512 \             # Relative weight (default 1024)
  myimage:latest
```

Explanation:

--memory="512m" → Container can use max 512 MB RAM.

--memory-reservation="256m" → Docker tries to keep 256 MB always available for this container, but it can exceed if host allows.

--cpus="1.5" → Limit container to 1.5 CPUs worth of time.

--cpu-shares=512 → Relative weight (e.g., if one container is 512 and another 1024, the second gets 2× CPU priority when competing).

Resource Management

Resource Management (in Docker Compose)

```
version: "3.9"
services:
  web:
    image: flask-ml-app:latest
    ports:
      - "5000:5000"
    deploy:
      resources:
        limits:
          cpus: "1.5"
          memory: 512M
        reservations:
          memory: 256M
```

Note:
deploy.resources works fully in Swarm mode. If you just run docker-compose up, the limits for memory still work, but CPU limits may need Swarm/Kubernetes.

To make it work:
Option 1: docker compose --compatibility up
Option 2: Use legacy fields directly

```
version: "3.9"
services:
  web:
    image: flask-ml-app:latest
    ports:
      - "5000:5000"
    mem_limit: 512m
    cpus: 1.5
```

Then run normally: docker compose up

Disclaimer

- Content of this presentation is not original and it has been prepared from various sources for teaching purpose.